

Nat Curtis

ME 3313

9/10/2025

ME 3313 Honors Project

Introduction

For this project, I am analyzing the linkages of a skid-steer bucket loader, shown in Figure 1. I will explore each of the topics we explore in ME 3313, starting with mobility, and going on to topics including range of motion, position, velocity, acceleration, and forces, and use each of these topics to analyze the linkage. I will then explore synthesis and try to create my own linkage for the bucket loader.



Figure 1: Side view of a Skid

Mobility

Mobility is a measure of how many degrees of freedom a linkage has, and it is calculated with Equation (1), called the Grubler-Kutzbach Mobility equation, where M is the mobility, L is the number of links, J_1 is the number of full joints, and J_2 is the number of half joints.

$$M = 3(L - 1) - 2J_1 - J_2 \quad (1)$$

A link is a single rigid body. A full joint is a joint with 1 degree of freedom, like a pin or two flat surfaces sliding along each other. A half joint is a joint with 2 degrees of freedom, like a pin in a joint, or two curved surface rolling or slipping along each other. On a kinematic diagram of a linkage, each link is marked with a unique plain number from 1 to the number of links, each full joint is marked with a unique circled number from 1 to the number of full joints, and each half joint is marked with a unique boxed number from 1 to the

number of half joints. As shown in Figure 2, the main linkage of the skid-steer has 6 links, 7 full joints, and 0 half joints. Using the mobility equation as shown in Equation (2), the mobility is calculated to be 1.

$$M = 3(L - 1) - 2J_1 - J_2 = 3(6 - 1) - 2 * 7 - 0 = 1 \quad (2)$$

Because the mobility is 1, only one input is needed to control the linkage. In this case, the input is a hydraulic piston, which is a combination of links 4 and 6. Figure 3 shows the annotated linkage diagram overlain on the original image of the skid-steer.

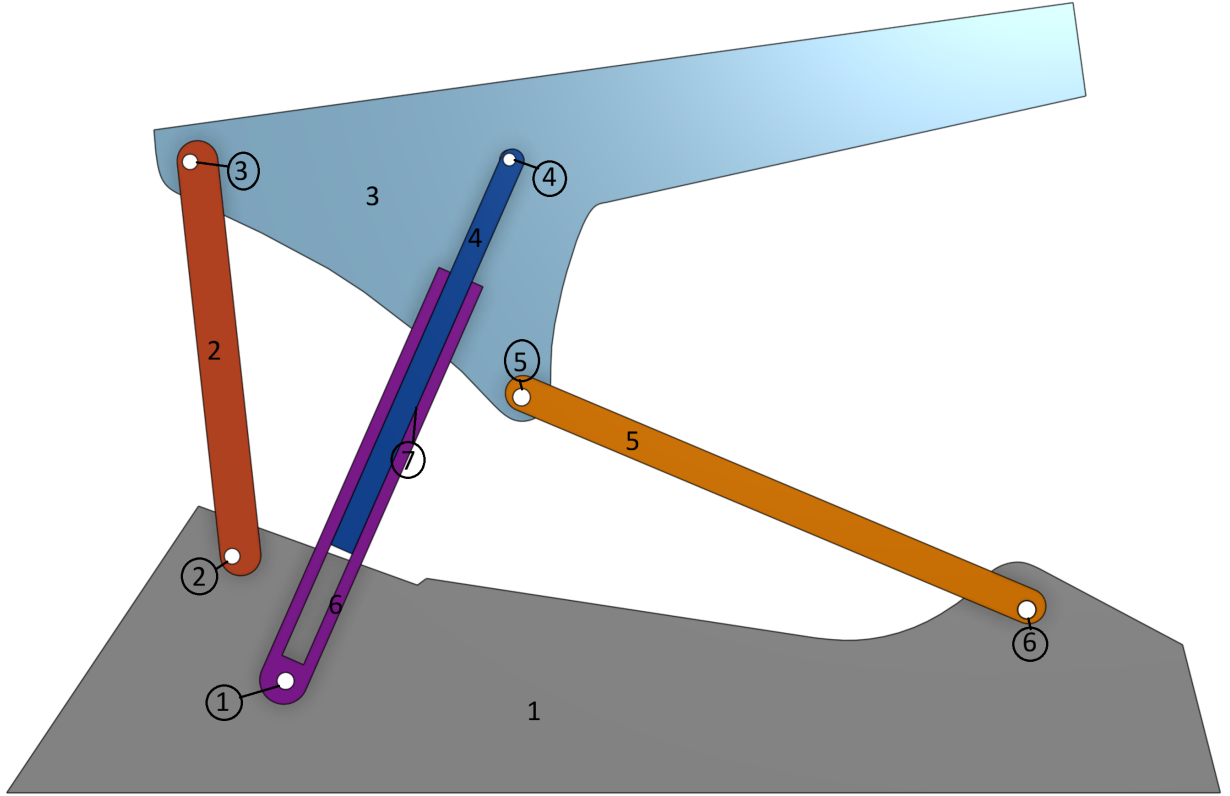


Figure 2: Annotated Linkage



Figure 3: Annotated Linkage Overlain On Skid-Steer

Multiloop Closure Equations

Mobility allows us to determine the degrees of freedom of a linkage, but usually we also need to determine the positions and angles of the individual links. To do this, we can use a method called Multiloop closure. It consists of drawing vectors between joints of each link, then writing equation summing loops of vectors, because the RHS of the equation has to equal zero. This is because the last vector ends where the first vector starts. These vector equations can then be turned into their scalar forms, which can be combined with equations relating rigid geometry of the links, providing a system of equations to solve. When done correctly, the number of unknowns minus the number of scalar and rigid geometry equations should equal the mobility. For the examined linkage, the vectors and loops are labeled as shown in Figure 4. The vector equations are shown below as eqs. (3) and (4).

$$\vec{R}_{BA} + \vec{R}_{DB} - \vec{R}_{CA} - \vec{R}_{GC} - \vec{R}_{DG} = 0 \quad (3)$$

$$\vec{R}_{GC} + \vec{R}_{DG} - \vec{R}_{DE} - \vec{R}_{EF} - \vec{R}_{FC} = 0 \quad (4)$$

These equations can be expanded into their scalar forms as shown in eqs. (5) to (10) below, where each θ is measured counter-clockwise from the horizontal to the tail of the corresponding vector.

$$R_{BA} \cos \theta_{BA} + R_{DB} \cos \theta_{DB} - R_{CA} \cos \theta_{CA} - R_{GC} \cos \theta_{GC} - R_{DG} \cos \theta_{DG} = 0 \quad (5)$$

$$R_{BA} \sin \theta_{BA} + R_{DB} \sin \theta_{DB} - R_{CA} \sin \theta_{CA} - R_{GC} \sin \theta_{GC} - R_{DG} \sin \theta_{DG} = 0 \quad (6)$$

$$R_{GC} \cos \theta_{GC} + R_{DG} \cos \theta_{DG} - R_{DE} \cos \theta_{DE} - R_{EF} \cos \theta_{EF} - R_{FC} \cos \theta_{FC} = 0 \quad (7)$$

$$R_{GC} \sin \theta_{GC} + R_{DG} \sin \theta_{DG} - R_{DE} \sin \theta_{DE} - R_{EF} \sin \theta_{EF} - R_{FC} \sin \theta_{FC} = 0 \quad (8)$$

$$\theta_{GC} - \theta_{DG} = 0 \quad (9)$$

$$\theta_{DB} + \beta_D - \theta_{DE} = 0 \quad (10)$$

In these equations, R_{BA} , R_{DB} , R_{CA} , θ_{CA} , R_{GC} , R_{DE} , R_{EF} , R_{FC} , θ_{FC} , and β_D are known from the geometry of the linkage and do not change. This leaves θ_{BA} , θ_{DB} , θ_{GC} , R_{DG} , θ_{DG} , θ_{DE} , and θ_{EF} as unknowns for a total of 7 unknowns. Taking 7 unknowns minus 6 equations yields 1 degree of freedom, which agrees with the earlier mobility analysis. Also, R_{DG} will be the input because it is controlled by a hydraulic piston.

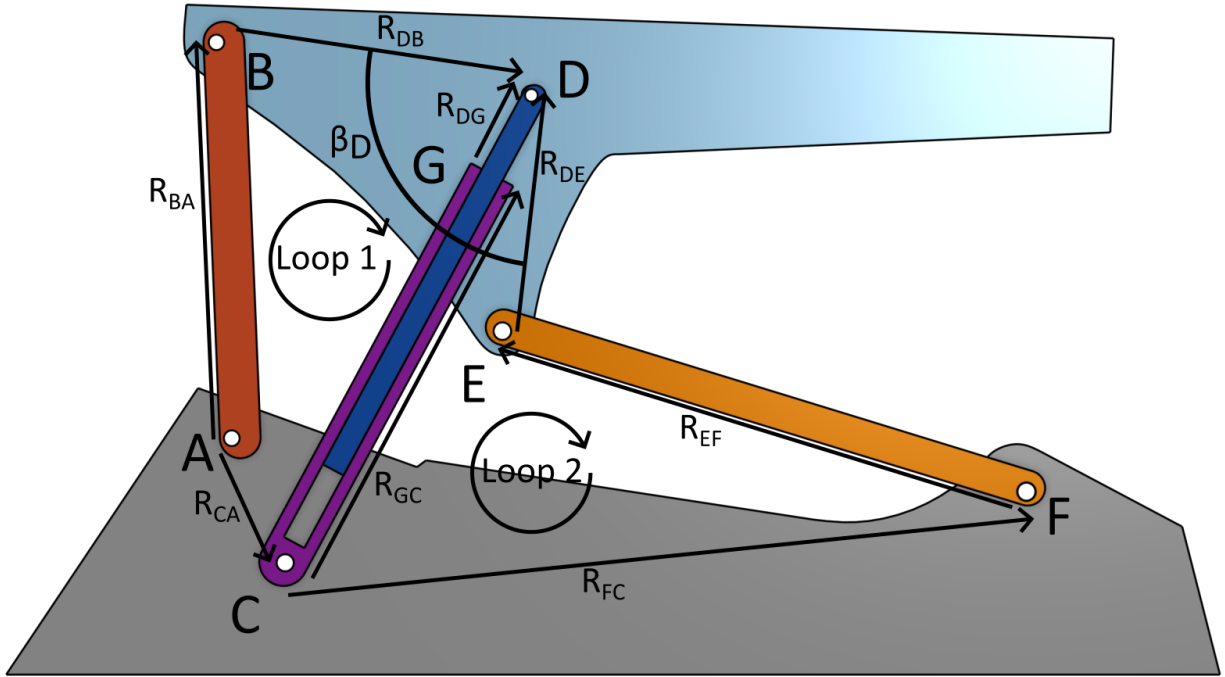


Figure 4: Diagram showing vector for Multiloop Equations

Loop Equation Derivatives

The next step in analyzing the linkage is to take the first and second derivatives of the scalar loop equations, which we can then use to find equations for velocity and acceleration of the links. The first derivatives of eqs. (5) to (10) are as shown below as eqs. (11) to (16), respectively.

$$-R_{BA}\omega_{BA} \sin \theta_{BA} - R_{DB}\omega_{DB} \sin \theta_{DB} + R_{GC}\omega_{GC} \sin \theta_{GC} - \dot{R}_{DG} \cos \theta_{DG} + R_{DG}\omega_{DG} \sin \theta_{DG} \quad (11)$$

$$R_{BA}\omega_{BA} \cos \theta_{BA} + R_{DB}\omega_{DB} \cos \theta_{DB} - R_{GC}\omega_{GC} \cos \theta_{GC} - \dot{R}_{DG} \sin \theta_{DG} - R_{DG}\omega_{DG} \cos \theta_{DG} \quad (12)$$

$$-R_{GC}\omega_{GC}\sin\theta_{GC} + \dot{R}_{DG}\cos\theta_{DG} - R_{DG}\omega_{DG}\sin\theta_{DG} + R_{DE}\omega_{DE}\sin\theta_{DE} + R_{EF}\omega_{EF}\sin\theta_{EF} \quad (13)$$

$$R_{GC}\omega_{GC}\cos\theta_{GC} + \dot{R}_{DG}\sin\theta_{DG} + R_{DG}\omega_{DG}\cos\theta_{DG} - R_{DE}\omega_{DE}\cos\theta_{DE} - R_{EF}\omega_{EF}\cos\theta_{EF} \quad (14)$$

$$\omega_{GC} - \omega_{DG} = 0 \quad (15)$$

$$\omega_{DB} - \omega_{DE} = 0 \quad (16)$$

The second derivatives of eqs. (5) to (10) are shown below as eqs. (17) to (22), respectively.

$$\begin{aligned} & -R_{BA}\alpha_{BA}\sin\theta_{BA} - R_{BA}\omega_{BA}^2\cos\theta_{BA} - R_{DB}\alpha_{DB}\sin\theta_{DB} - R_{DB}\omega_{DB}^2\cos\theta_{DB} + R_{GC}\alpha_{GC}\sin\theta_{GC} \\ & + R_{GC}\omega_{GC}^2\cos\theta_{GC} - \ddot{R}_{DG}\cos\theta_{DG} + 2\dot{R}_{DG}\omega_{DG}\sin\theta_{DG} + R_{DG}\alpha_{DG}\sin\theta_{DG} + R_{DG}\omega_{DG}^2\cos\theta_{DG} \end{aligned} \quad (17)$$

$$\begin{aligned} & R_{BA}\alpha_{BA}\cos\theta_{BA} - R_{BA}\omega_{BA}^2\sin\theta_{BA} + R_{DB}\alpha_{DB}\cos\theta_{DB} - R_{DB}\omega_{DB}^2\sin\theta_{DB} - R_{GC}\alpha_{GC}\cos\theta_{GC} \\ & + R_{GC}\omega_{GC}^2\sin\theta_{GC} - \ddot{R}_{DG}\sin\theta_{DG} - 2\dot{R}_{DG}\omega_{DG}\cos\theta_{DG} - R_{DG}\alpha_{DG}\cos\theta_{DG} + R_{DG}\omega_{DG}^2\sin\theta_{DG} \end{aligned} \quad (18)$$

$$\begin{aligned} & -R_{GC}\alpha_{GC}\sin\theta_{GC} - R_{GC}\omega_{GC}^2\cos\theta_{GC} + \ddot{R}_{DG}\cos\theta_{DG} - 2\dot{R}_{DG}\omega_{DG}\sin\theta_{DG} - R_{DG}\alpha_{DG}\sin\theta_{DG} \\ & - R_{DG}\omega_{DG}^2\cos\theta_{DG} + R_{DE}\alpha_{DE}\sin\theta_{DE} + R_{DE}\omega_{DE}^2\cos\theta_{DE} + R_{EF}\alpha_{EF}\sin\theta_{EF} + R_{EF}\omega_{EF}^2\cos\theta_{EF} \end{aligned} \quad (19)$$

$$\begin{aligned} & R_{GC}\alpha_{GC}\cos\theta_{GC} - R_{GC}\omega_{GC}^2\sin\theta_{GC} + \ddot{R}_{DG}\sin\theta_{DG} + 2\dot{R}_{DG}\omega_{DG}\cos\theta_{DG} + R_{DG}\alpha_{DG}\cos\theta_{DG} \\ & - R_{DG}\omega_{DG}^2\sin\theta_{DG} - R_{DE}\alpha_{DE}\cos\theta_{DE} + R_{DE}\omega_{DE}^2\sin\theta_{DE} - R_{EF}\alpha_{EF}\cos\theta_{EF} + R_{EF}\omega_{EF}^2\sin\theta_{EF} \end{aligned} \quad (20)$$

$$\alpha_{GC} - \alpha_{DG} = 0 \quad (21)$$

$$\alpha_{DB} - \alpha_{DE} = 0 \quad (22)$$

Code Introduction

To solve these equations, I used Python code taking advantage of the `fsolve` function from the SciPy library. This is a numerical solver function that takes in a set of equations and initial guesses and outputs a solution. Depending on the initial guess, this solution can be incorrect, or a valid solution but not the correct solution for the actual problem constraints. To increase the chances significantly of a valid solution being generated, I generated a range of initial condition options for each unknown variable and then generated a solution for every combination of initial conditions. Taking these, I kept only ones that had little error when plugged back into the multiloop equations and each variable was within an estimated range of motion. The known link lengths that were plugged into the equations are shown below, and the input length is shown under that. These measurements are only estimates, however, because I found them by finding the length of the loader in its documentation, then measured the length in a side view image of the loader. I took the

ratio of these, then measured each link length and and angle in the picture and multiplied each link length by the ratio to get the estimated actual link length. Angles however are not affected by scaling an image up or down though, so I used the measured angles as they were.

I then took this solution and used it to find solutions along a range of input angles. Because angles that are close together will have similar solutions, I was able to start at my calculated solution and use it as the initial guess for a slightly different angle. This would quickly converge to another solution close by, yielding a correct solution. Using this new solution as my next guess, I was able to repeat this and get the solution for a range of input angles. I then wrote equations for the absolute position of each joint in the linkage, with the origin at Joint C. The vector forms of these equations are shown below in eqs. (23) to (30) (Note some valu). Using these equations and the range of solutions I found previously I plotted the path each joint followed for that range of angles, shown in Figure 5.

$$\vec{A} = -R_{CA} \quad (23)$$

$$\vec{B} = -R_{CA} + R_{BA} \quad (24)$$

$$\vec{C} = 0 \quad (25)$$

$$\vec{D} = \vec{E} = R_{FC} + R_{EF} + R_{DE} \quad (26)$$

$$\vec{E} = R_{FC} + R_{EF} \quad (27)$$

$$\vec{F} = R_{FC} \quad (28)$$

$$\vec{G} = R_{GC} \quad (29)$$

$$\vec{P} = -R_{CA} + R_{BA} + R_{PB} \quad (30)$$

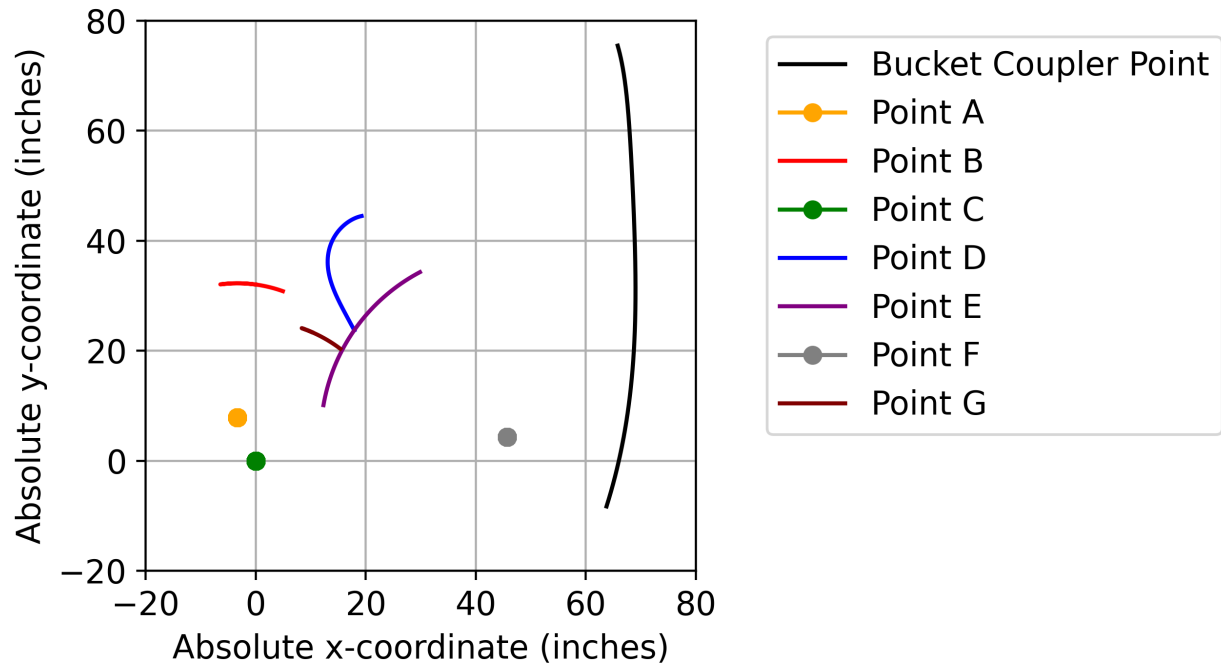


Figure 5: Path of absolute position of linkage joints

Link Measurements

$$R_{BA} = 24.4 \text{ in.}$$

$$R_{DB} = 19.8 \text{ in.}$$

$$R_{CA} = 8.5 \text{ in.}$$

$$\theta_{CA} = 293.04^\circ$$

$$R_{GC} = 25.5 \text{ in.}$$

$$R_{DE} = 14.7 \text{ in.}$$

$$R_{EF} = 33.9 \text{ in.}$$

$$R_{FC} = 45.9 \text{ in.}$$

$$\theta_{FC} = 5.44^\circ$$

$$\beta_D = 92.47^\circ$$

Input Link Length

$$R_{DC} = 29.72 \text{ in.}$$

Multiloop Solver Code

```
1 import os
2 import numpy as np
3 import math
4 from scipy.optimize import fsolve
5 import matplotlib.pyplot as plt
6 from warnings import catch_warnings
7
8 act_img_ratio = 0.1718 # Approximate ratio of length without bucket from specs
9     ↪ in inches over length without bucket from image in mm
10
11 #act_img_ratio = 1      # Uncomment to use image link lengths instead of
12     ↪ approximated actual lengths
13 # Known values (For the moment just measured in millimeters from image
14     ↪ multiplied by ratio)
15 rba = 142.27*act_img_ratio
16 rdb = 115.2*act_img_ratio
17 rca = 49.46*act_img_ratio
18 thetaca = math.radians(293.04)
19 rgc = 148.5*act_img_ratio # Can't really tell from picture, but should not
20     ↪ affect results as long as counted for accordingly
21 rde = 85.65*act_img_ratio
22 ref = 197.12*act_img_ratio
23 rfc = 267.12*act_img_ratio
24 thetafc = math.radians(5.44) # angle from positive x-axis to ground link, which
25     ↪ is Rea
26 betad = math.radians(92.47) # Angle from DB to DA
27 rpb = 439.41*act_img_ratio # distance from point b to the bucket coupler point
28 betab = math.radians(7.41) # Angle from rpb to rdb
29
30 # Inputs
31 rdc = 173*act_img_ratio
32
33 #print(rba, rdb, rca, rgc, rde, ref, rfc, rdc)
34 rdg = rdc - rgc # Given input length
35
36 # Finds most common element in a list
37 def most_common(lst: list):
38     return max(set(lst), key=lst.count)
39
40 """
41 Unknowns:
42 x[0]: thetaba
43 x[1]: thetadb
44 x[2]: thetagc
45 x[3]: thetadg
```



```

43 x[4]: thetade
44 x[5]: thetaef
45 """
46 unknown_length_indices = []
47
48 num_equations = 6
49
50 # System of scalar equations to solve
51 def eqs(x):
52     return [
53
54         rba*np.cos(x[0]) + rdb*np.cos(x[1]) - rca*np.cos(thetaca) -
55         ↪ rgc*np.cos(x[2]) - rdg*np.cos(x[3]),
56         rba*np.sin(x[0]) + rdb*np.sin(x[1]) - rca*np.sin(thetaca) -
57         ↪ rgc*np.sin(x[2]) - rdg*np.sin(x[3]),
58         rgc*np.cos(x[2]) + rdg*np.cos(x[3]) - rde*np.cos(x[4]) -
59         ↪ ref*np.cos(x[5]) - rfc*np.cos(thetafc),
60         rgc*np.sin(x[2]) + rdg*np.sin(x[3]) - rde*np.sin(x[4]) -
61         ↪ ref*np.sin(x[5]) - rfc*np.sin(thetafc),
62         x[2]-x[3],
63         x[1]+betad-x[4]
64     ]
65
66 # Rename pi for shorter call
67 pi = math.pi
68
69 # For finding right guesses
70 initial_guess_min = [0, 0, 0, 0, 0, 0]
71 initial_guess_max = [2*pi, 2*pi, 2*pi, 2*pi, 2*pi, 2*pi]
72 current_guess = [0, 0, 0, 0, 0, 0]
73
74 # Create empty set to store unique solutions
75 gen_solutions = set()
76
77 # Check if solution is in approximate range of motion
78 def result_check(result) -> bool:
79     if result[0] > 3*pi/4 or result[0] < pi/4:
80
81         return False
82     if result[1] > pi/2 and result[1] < 3*pi/2:
83
84         return False
85     if result[2] > pi/2:
86
87         return False
88     if result[3] > pi/2:
89
90         return False
91     if result[4] > pi:
92
93         return False
94     if result[5] < pi/2 or result[5] > pi:
95
96         return False
97     return True

```

```

94
95 # Check error of solution
96 """def calc_err(result, eq=eqs):
97     err = eq(result)
98     err_sum = 0
99     for i in range(len(err)):
100         err_sum += float(np.abs(err[i]))
101     return err_sum"""
102
103 def calc_err(result, eq=eqs, unkn_len_ind=unknown_length_indices):
104     err = eq(result)
105     err_sum = 0
106     for i in range(len(err)):
107         if i in unkn_len_ind:
108             calc_err_amt = float(np.abs(err[i]))
109         else:
110             calc_err_amt = float(np.abs(err[i])) % 2*pi
111             calc_err_amt = min(calc_err_amt, 2*pi - calc_err_amt)
112         err_sum += calc_err_amt
113     return err_sum
114
115
116 # Populate the solution set with valid solutions
117 def solution_gen(current_guess: list[float] | None = None, guess_per_var: int =
    ↳ 5, index: int = 0, initial_min: list[float] = initial_guess_min,
    ↳ initial_max: list[float] = initial_guess_max, output: set[tuple[float, ...]]
    ↳ = gen_solutions, eq=eqs, num_var: int = num_equations,
    ↳ unkn_len_ind=unknown_length_indices):
118     if current_guess is None:
119         current_guess = []
120     if len(initial_min) != num_var and len(initial_max) != num_var:
121         exit("Initial and/or final guess are wrong length")
122
123     if index == num_var:
124         with catch_warnings(record=True):
125             sol = fsolve(eq, current_guess)
126             curr_error = calc_err(sol, eq)
127             if result_check(sol) and curr_error < 0.1:
128                 output.add(tuple(sol))
129     return
130
131     guesses = np.linspace(initial_min[index], initial_max[index], guess_per_var)
132
133     # Finds solutions with all different combinations of guesses
134     for guess in guesses:
135         solution_gen(current_guess=current_guess+[guess],
            ↳ guess_per_var=guess_per_var, index=index+1, initial_min=initial_min,
            ↳ initial_max=initial_max, output=output, eq=eq, num_var=num_var,
            ↳ unkn_len_ind=unkn_len_ind)
136     return
137
138
139 # Absolute position of bucket coupler point with poiny C as the origin, positive
    ↳ x-axis to the right
140 def link_point_pos(angles):

```

```

141
142     point_c = (0, 0)
143
144     point_a = (-rca*math.cos(thetaca), -rca*math.sin(thetaca))
145
146     point_f = (rfc*math.cos(thetafc), rfc*math.sin(thetafc))
147
148     point_b = (point_a[0] + rba*math.cos(angles[0]), point_a[1] +
149 ↪      rba*math.sin(angles[0]))
150
151     point_e = (point_f[0] + ref*math.cos(angles[5]), point_f[1] +
152 ↪      ref*math.sin(angles[5]))
153
154     point_g = (rgc*math.cos(angles[2]), rgc*math.sin(angles[2]))
155
156     point_d = (point_e[0] + rde*math.cos(angles[4]), point_e[1] +
157 ↪      rde*math.sin(angles[4]))
158
159     bucket = (point_b[0] + rpb*math.cos(angles[1]-betab), point_b[1] +
160 ↪      rpb*math.sin(angles[1]-betab))
161
162     return point_a, point_b, point_c, point_d, point_e, point_f, point_g, bucket
163
164 # Generate solutions
165 solution_gen()
166
167 # Turn set into a list so it is indexable
168 gen_solutions = list(gen_solutions)
169
170 min_error = 1
171 min_sol = None
172 for sol in gen_solutions:
173     err = calc_err(sol)
174     if err < min_error:
175         min_error = err
176         min_sol = sol
177
178 s = min_sol
179
180 if s is None:
181     exit("No solutions found")
182
183 sol_domain = np.linspace(rdc, 1.9*rgc, 10000)
184 rdc_store = rdc
185 sol_set = [s]
186 for x in sol_domain[1:]:
187     rdc = x
188     rdg = rdc - rgc
189     with catch_warnings(record=True):
190         new_sol = fsolve(eqs, sol_set[-1])
191         sol_set.append(new_sol)

```

```

192
193 rdc = rdc_store
194 rdg = rdc - rgc
195 #print(sol_set)
196
197 point_ax = []
198 point_ay = []
199 point_bx = []
200 point_by = []
201 point_cx = []
202 point_cy = []
203 point_dx = []
204 point_dy = []
205 point_ex = []
206 point_ey = []
207 point_fx = []
208 point_fy = []
209 point_gx = []
210 point_gy = []
211 bucket_x = []
212 bucket_y = []
213 for i in range(len(sol_set)):
214     point_pos = link_point_pos(sol_set[i])
215     point_ax.append(point_pos[0][0])
216     point_ay.append(point_pos[0][1])
217     point_bx.append(point_pos[1][0])
218     point_by.append(point_pos[1][1])
219     point_cx.append(point_pos[2][0])
220     point_cy.append(point_pos[2][1])
221     point_dx.append(point_pos[3][0])
222     point_dy.append(point_pos[3][1])
223     point_ex.append(point_pos[4][0])
224     point_ey.append(point_pos[4][1])
225     point_fx.append(point_pos[5][0])
226     point_fy.append(point_pos[5][1])
227     point_gx.append(point_pos[6][0])
228     point_gy.append(point_pos[6][1])
229     bucket_x.append(point_pos[7][0])
230     bucket_y.append(point_pos[7][1])
231
232
233
234
235 def r_degrs(num):
236     return num/pi*180 % 360
237
238 s = list(s)
239 for i in range(len(s)):
240     if not i in unknown_length_indices:
241         s[i] = r_degrs(s[i])
242 print(f"Theta_BA = {s[0]} degrees")
243 print(f"Theta_DB = {s[1]} degrees")
244 print(f"Theta_GC = {s[2]} degrees")
245 print(f"Theta_DG = {s[3]} degrees")
246 print(f"Theta_DE = {s[4]} degrees")

```

```

247 print(f"Theta_EF = {s[5]} degrees")
248
249 plot_path =
    ↪ os.path.join(os.path.split(os.path.split(os.path.abspath(__file__))[0])[0],
    ↪ "imgs", "LinkPointCoordinates.png")
250
251 # Link Points Plot
252 plt.rcParams['font.size'] = 12
253 fig1, ax1 = plt.subplots(layout='constrained')
254 ax1.grid()
255 ax1.plot(bucket_x, bucket_y, 'black', label = 'Bucket Coupler Point')
256 ax1.plot(point_ax, point_ay, color='orange', marker='o', label = 'Point A')
257 ax1.plot(point_bx, point_by, 'red', label = 'Point B')
258 ax1.plot(point_cx, point_cy, color='green', marker='o', label = 'Point C')
259 ax1.plot(point_dx, point_dy, 'blue', label = 'Point D')
260 ax1.plot(point_ex, point_ey, 'purple', label = 'Point E')
261 ax1.plot(point_fx, point_fy, color='gray', marker='o', label = 'Point F')
262 ax1.plot(point_gx, point_gy, 'maroon', label = 'Point G')
263 ax1.set_xlim(-20, 80)
264 ax1.set_ylim(-20, 80)
265 ax1.set_aspect('equal', adjustable='box')
266 ax1.set_xlabel('Absolute x-coordinate (inches)')
267 ax1.set_ylabel('Absolute y-coordinate (inches)')
268 ax1.legend(bbox_to_anchor=(1.1, 1), loc='upper left')
269 plt.savefig(plot_path, dpi=400)

```

Multiloop Solver Output

```

PS C:\... \Honors> python code/main.py
Theta_BA = 82.2 degrees
Theta_DB = 335.1 degrees
Theta_GC = 52.9 degrees
Theta_DG = 52.9 degrees
Theta_DE = 67.6 degrees
Theta_EF = 170.2 degrees

```
